

Basic (a simple version log) — Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Story — `version_log_practice.py` : a release gate needs something to compare against, and "which prompt was live last Tuesday?" needs a record. This log is that record — every version saved with its text and time, never overwritten. **If not:** the Build Task's gate would have nothing to check against, and a bad answer couldn't be traced back to the prompt that produced it.

All examples below use a stand-in for Doc13's real test suite, `run_eval_suite(prompt_text)` , so the script runs on its own. In your real Project 5 code, swap it for your actual Doc13 suite import.

Basic Version

Approach 1 — the direct way

Story: save two versions and score each one — the smallest thing that makes "which version is better?" answerable. **If not:** you'd compare prompts from memory.

```
# version_log_practice.py
import json
import logging
from datetime import datetime

logging.basicConfig(level=logging.INFO)

def run_eval_suite(prompt_text):
    if "politely" in prompt_text:
        return {"score": 1.0}
    return {"score": 0.6}

versions = []
versions.append({
    "version_name": "v1",
    "prompt_text": "Answer the customer's question.",
    "created_at": str(datetime.now()),
})
versions.append({
    "version_name": "v2",
    "prompt_text": (
```

```

        "Answer the customer's question politely and cite the source."
    ),
    "created_at": str(datetime.now()),
})

with open("versions.json", "w") as f:
    json.dump(versions, f, indent=2)
logging.info("Saved " + str(len(versions)) + " versions to versions.json")

for version in versions:
    result = run_eval_suite(version["prompt_text"])
    print(version["version_name"], result["score"])

```

Expected output:

```

INFO:root:Saved 2 versions to versions.json
v1 0.6
v2 1.0

```

This version works correctly for what the exercise asks. It rebuilds `versions.json` from scratch every run (no loading of prior entries first) and uses `str(datetime.now())` for a quick, locally-formatted timestamp — both fine for a first working version.

Revision from Doc01 (Basic): saving the log is reported with `logging.info(...)`, the simplest Doc01 setup — one `logging.basicConfig(...)` line, then one call. `INFO` is the right level: writing the file is normal progress, nothing went wrong. The scores stay as `print(...)`, because they are the result you asked the script for; the "saved" line is a note about *what the script did*, which is exactly what logging is for.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — reusable save/load functions

Story: a log you rewrite from scratch each run isn't a history. Load first, then append, and stop loudly if the file is damaged. **If not:** a half-written file would quietly become an empty list, and the next save would wipe every version.

```
# version_log_practice.py
import json
import logging
from datetime import datetime, timezone
from pathlib import Path

logger = logging.getLogger(__name__)
logger.setLevel(logging.INFO)
logger.addHandler(logging.StreamHandler())

class VersionLogCorruptError(Exception):
    pass

def run_eval_suite(prompt_text: str) -> dict:
    if "politely" in prompt_text:
        return {"score": 1.0}
    return {"score": 0.6}

def load_versions(path: str) -> list[dict]:
    if not Path(path).exists():
        return []
    with open(path) as f:
        try:
            return json.load(f)
        except json.JSONDecodeError:
            # why: a damaged file must stop the save – silently treating
            # it as [] would wipe the whole history on the next write
            raise VersionLogCorruptError(
                f"{path} is not valid JSON – restore it before saving, "
                "or its history will be lost"
            )

def save_version(name: str, prompt_text: str, path: str) -> None:
```

```

# how: load what's already there first, then append – never
# rebuild the file from scratch
versions = load_versions(path)
versions.append({
    "version_name": name,
    "prompt_text": prompt_text,
    "created_at": datetime.now(timezone.utc).isoformat(),
})
with open(path, "w") as f:
    json.dump(versions, f, indent=2)
logger.info(f"Saved version {name} ({len(versions)} in log)")

save_version("v1", "Answer the customer's question.", "versions.json")
save_version(
    "v2",
    "Answer the customer's question politely and cite the source.",
    "versions.json",
)

for version in load_versions("versions.json"):
    result = run_eval_suite(version["prompt_text"])
    print(version["version_name"], result["score"])

```

Expected output:

```

Saved version v1 (1 in log)
Saved version v2 (2 in log)
v1 0.6
v2 1.0

```

(the log lines show only the message, because this handler has no formatter)

And if `versions.json` is damaged — say, half-written when the script was stopped — `save_version` now stops instead of overwriting it. The last line of the traceback:

```

__main__.VersionLogCorruptError: versions.json is not valid JSON – restore it
before saving, or its history will be lost

```

Revision from Doc01 (Intermediate): two habits come back here. First, a named logger, `logging.getLogger(__name__)`, instead of the root logger — so these lines say which module they came from once this code is imported by the release gate. Second, catching one *specific* exception, `json.JSONDecodeError`, and raising a named error with a clear message. This matters a lot here: `save_version` loads, then rewrites the whole file. If a broken file quietly became `[]`,

the next save would wipe the whole version history. The named error stops that before any damage is done.

Approach 2 — same idea, printed as a readable table

Story: the same data, lined up so v1 and v2 can be compared at a glance. **If not:** you'd squint at separate print lines to spot the better version.

```
# version_log_practice.py
def print_score_table(versions: list[dict]) -> None:
    print(f"{'version':<10}{'score':<8}{'created_at'}")
    for version in versions:
        result = run_eval_suite(version["prompt_text"])
        name = version["version_name"]
        score = result["score"]
        print(f"{name:<10}{score:<8}{version['created_at']}")

print_score_table(load_versions("versions.json"))
```

Expected output:

version	score	created_at
v1	0.6	2026-09-11T14:03:00.123456+00:00
v2	1.0	2026-09-11T14:03:00.234567+00:00

Difference from Basic: both approaches add full type hints, a named logger, a `VersionLogCorruptError` for a damaged file, and split the work into `save_version` / `load_versions` functions that *load before they save*, so calling `save_version` twice in a row (once for v1, once for v2) builds up one growing list in `versions.json` instead of the Basic Version's approach of building the whole list in memory first and writing it once. The timestamp also switches to `datetime.now(timezone.utc).isoformat()` — a sortable, unambiguous string, instead of `str(datetime.now())`'s locale-dependent format. Approach 2 is a pure formatting improvement — same data, read as a table instead of separate print lines.

Which one should you actually write? Approach 1 — load-then-append with a named error for a damaged file — is the version the Build Task's `versioning.py` builds on. Approach 2's table is worth adding once you have more than two or three versions to compare.